MADL Experiment 9

Campus Connect Flutter and Firebase

Name: Parth Chaudhari **Roll No:** 5024112
Date: 12 October 2026

Aim

To connect a Flutter user interface to Firebase Realtime Database and demonstrate student record operations through a web application.

Objective

To develop Campus Connect, a student directory that supports adding, reading, searching, filtering, and deleting student records, with realtime updates and authenticated database access.

Software and Services

Flutter 3.47.2, Dart 3.13.2, Firebase Core, Firebase Authentication, Firebase Realtime Database, and a web browser. Android configuration is also included in the project.

Detailed Steps

1. Select the Firebase project
Use the existing Campus Connect project with Project ID **campus-connect-123da**. Register the web app and the Android package **com.example.campus_connect**.

2. Configure the Realtime Database
Use the active database in us-central1. Store each student below **users/<uid>/students/<push-id>**. Firebase generates a unique record ID when push() is called.

3. Enable authentication
Enable Anonymous sign-in. The app signs in automatically and obtains a UID before accessing student records.

4. Configure database access
Deploy database.rules.json. Allow a signed-in user to access only the node matching their UID, and validate the student fields.

Detailed Steps Continued

5. Connect Flutter to Firebase
Add firebase_core, firebase_auth, and firebase_database. Save the registered app configuration in lib/firebase_options.dart and Android google-services.json. Initialize Firebase at startup.

6. Create the Flutter interface
Build a responsive Material 3 interface with full name, email, department, and an optional note. Provide Save student, Read records, search, department filters, and a delete action on each card.

7. Perform database operations
SAVE uses ref.push().set(). READ uses ref.get(), while ref.onValue keeps the directory current. DELETE uses ref.child(id).remove() after the confirmation dialog.

8. Run and demonstrate the application
From the campus_connect folder, run flutter pub get and flutter run -d chrome. Enter valid student details and click Save student. Use Read records to fetch the stored records.

9. Verify cloud persistence
Inspect the Realtime Database under users/<uid>/students. Reload the application to verify that saved records remain available. The demo records were also retrieved independently using the Firebase CLI.

Database Structure

Field	Type	Purpose
push-id	Generated string key	Uniquely identifies each student
name	String, 2 to 80 characters	Student full name
email	String, up to 120 characters	Student email address
department	String from allowed list	Student department
note	String, up to 300 characters	Optional student note
createdAt	Server timestamp	Sort records by creation time

Application Flow

Flutter form -> Firebase initialization -> Anonymous authentication -> UID-scoped database reference -> write or read operation -> realtime student cards.

Database URL
https://campus-connect-123da-default-rtdb.firebaseio.com

Code Firebase Initialization and Save

The following excerpts are from the implemented Flutter application. The complete interface is available in campus_connect/lib/main.dart.

```
import 'package:firebase_core/firebase_core.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'package:firebase_database/firebase_database.dart';
import 'firebase_options.dart';

Future<DatabaseReference> connect() async {
    final app = Firebase.apps.isEmpty
        ? await Firebase.initializeApp(
            options: DefaultFirebaseOptions.currentPlatform)
        : Firebase.app();
    const url = String.fromEnvironment('DATABASE_URL');
    final db = url.isEmpty
        ? FirebaseDatabase.instance
        : FirebaseDatabase.instanceFor(app: app, databaseURL: url);
    final auth = FirebaseAuth.instance;
    final user = auth.currentUser ?? (await auth.signInAnonymously()).user;
    if (user == null) {
      throw StateError('Sign-in did not complete. Please retry.');
    }
    return db.ref('users/${user.uid}/students');
  }

Future<void> submit() async {
    if (!form.currentState!.validate()) return;
    FocusScope.of(context).unfocus();
    setState(() => saving = true);
    try {
      await widget.ref.push().set({
        'name': name.text.trim(),
        'email': email.text.trim(),
        'department': department,
        'note': note.text.trim(),
        'createdAt': ServerValue.timestamp
      });
      if (!mounted) return;
      name.clear();
      email.clear();
      note.clear();
      form.currentState!.reset();
      message('Student saved to Firebase.');
    } catch (e) {
      message('Could not save: $e');
    } finally {
      if (mounted) setState(() => saving = false);
    }
  }
```

Code Read and Delete

```
Future<void> readRecords() async {
    setState(() => reading = true);
    try {
      final snapshot = await widget.ref.get();
      message(
          'Read ${snapshot.children.length} student records from Firebase.');
    } catch (e) {
      message('Could not read records: $e');
    } finally {
      if (mounted) setState(() => reading = false);
    }
  }

Future<void> remove(String id) async {
    final confirmed = await showDialog<bool>(
        context: context,
        builder: (context) => AlertDialog(
                title: const Text('Delete this student?'),
                content:
                    const Text('This removes the record from your database.'),
                actions: [
                  TextButton(
                      onPressed: () => Navigator.pop(context, false),
                      child: const Text('Cancel')),
                  FilledButton(
                      onPressed: () => Navigator.pop(context, true),
                      child: const Text('Delete'))
                ]));
    if (confirmed != true || !mounted) return;
    try {
      await widget.ref.child(id).remove();
      message('Student deleted.');
    } catch (e) {
      message('Could not delete: $e');
    }
  }
```

Realtime Reading and Access Rules

```
records = widget.ref.onValue;
// StreamBuilder<DatabaseEvent> rebuilds the list from this stream.

// database.rules.json: access is scoped to the signed-in UID.
{
  "rules": {
    ".read": false,
    ".write": false,
    "users": {
      "$uid": {
        ".read": "auth != null && auth.uid === $uid",
        ".write": "auth != null && auth.uid === $uid"
      }
    }
  }
}
// The deployed rules also validate all student fields.
```

Output

Actual screenshots of Campus Connect connected to the Firebase project. The displayed student names and example.com addresses are fictional demonstration data.

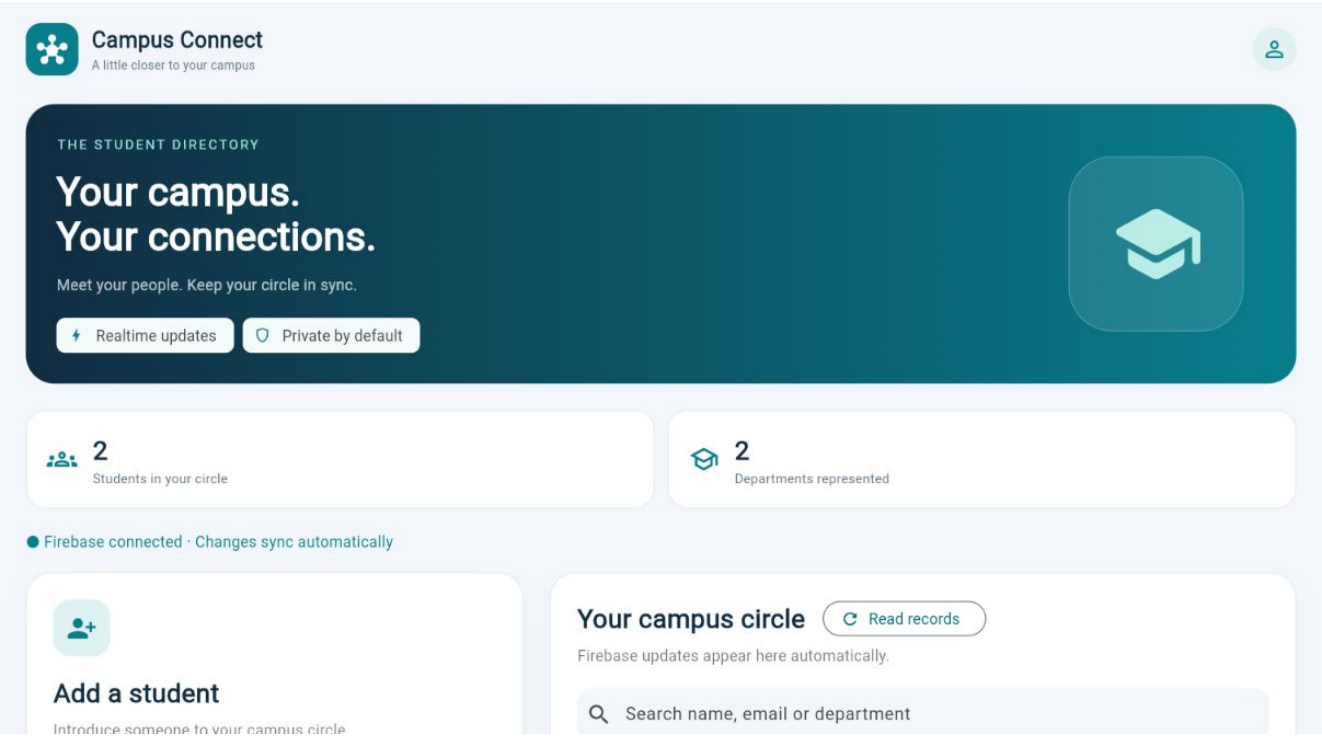


Figure 1 Application dashboard with Firebase connected and two saved students

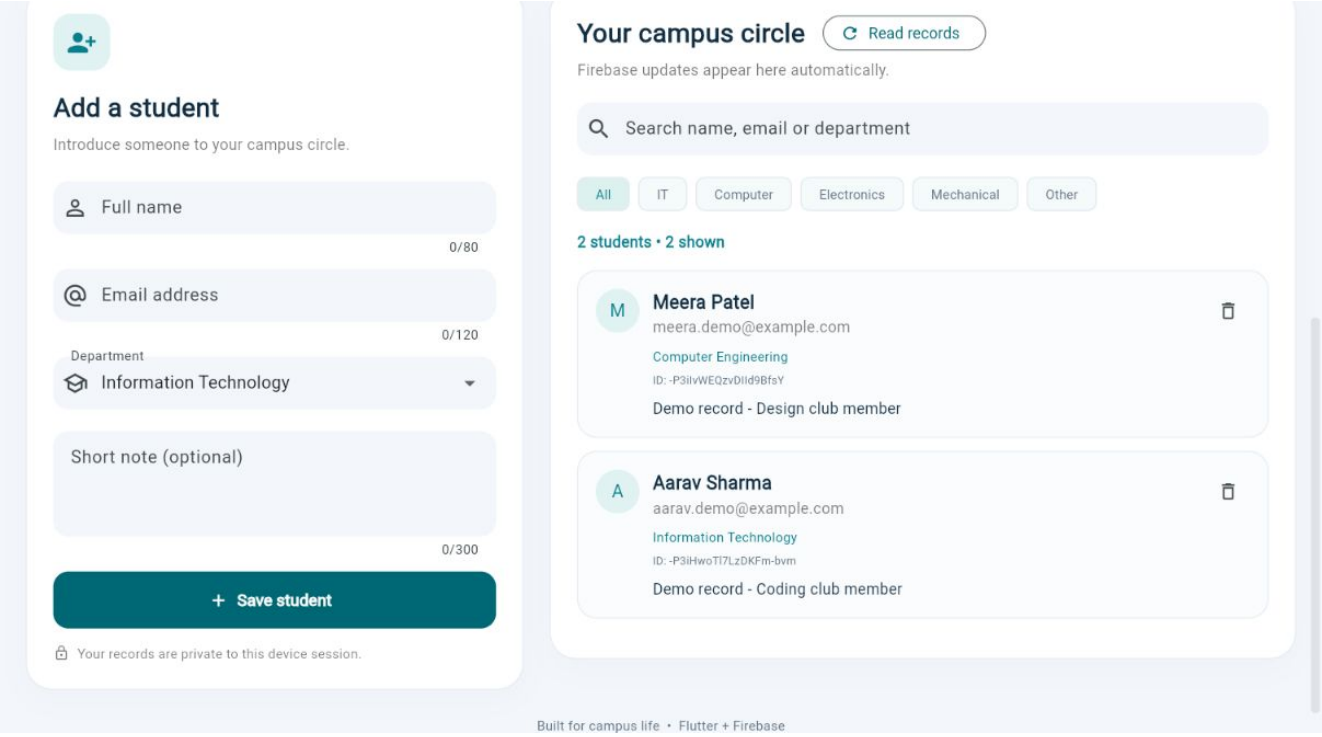


Figure 2 Input form and student records retrieved from the live database

Observation and Verification

Check	Observed result
Firebase initialization and authentication	Connected successfully; Firebase connected indicator displayed
Save and realtime display	Two fictional records saved and displayed automatically
Explicit read	Read records returned 2 student records from Firebase
Cloud persistence	Both records retrieved with the Firebase CLI and shown after reloading
Delete operation	Implemented with user confirmation; not executed in this demo
Static analysis	flutter analyze: No issues found
Setup screen test	flutter test test/setup_test.dart: All tests passed
Web compilation	flutter build web --no-web-resources-cdn: Successful

Stored Demo Records

Name	Email	Department
Aarav Sharma	aarav.demo@example.com	Information Technology
Meera Patel	meera.demo@example.com	Computer Engineering

Conclusion

Campus Connect was successfully connected to Firebase Authentication and Realtime Database. Student records were saved, retrieved, persisted, and displayed through the Flutter interface with realtime updates. The application also implements confirmed deletion, form validation, search, department filtering, and recoverable error states.

Each anonymous session accesses its own records. Clearing browser storage or reinstalling the app may create a new UID and a separate directory.

References

https://firebase.google.com/docs/flutter/setup

https://firebase.google.com/docs/database/flutter/read-and-write

https://firebase.google.com/docs/auth/flutter/anonymous-auth

https://firebase.google.com/docs/database/security